Research Implementation Session

Algoverse AI Research Program: Spring 2026

ALGOVERSE
CODING ACADEMY

Today's Research Projects

Project 1: Do LLMs give consistent answers when you rephrase the same question?

- Design 20 questions, create 3 paraphrase variants each

- Test across 2-3 models

- Measure consistency (exact match, semantic similarity)

Project 2: Which prompting strategy works best for your chosen task?

- Pick a task (math, reasoning, classification, summarization)

- Test 3-4 prompting strategies (zero-shot, few-shot, CoT, CoT+SC)

- Compare accuracy across strategies and models

Both are real, possible research directions.

Why These Projects?

Skill	Benchmark project	Ablation experiment
Experiment design	Design evaluation categories	Design prompting conditions
Data curation	Create question + paraphrase sets	Curate test examples per task
API integration	Call multiple LLM APIs	Call multiple LLM APIs
Statistical analysis	Consistency metrics	Accuracy comparisons
Paper writing	Tables, figures, claims	Tables, figures, claims
Reproducibility	Fixed seed, versioned prompts	Fixed seed, controlled variables

Example Questions for the Benchmark

Category	Example Question	Paraphrase 1	Paraphrase 2
Math	"What is 15% of 200?"	"Calculate 15 percent of 200."	"200 × 0.15 = ?"
Logic	"If all roses are flowers and some flowers fade quickly, can we conclude some roses fade quickly?"	"Given every rose is a flower and certain flowers fade fast, does it follow that certain roses fade fast?"	"All roses ⊂ flowers. Some flowers fade. Must some roses?"
Factual	"What is the capital of Australia?"	"Which city serves as Australia's capital?"	"Name Australia's capital city."

Pick YOUR OWN categories, medical, legal, coding, history, psychology...

The more varied across the room, the better the discussion

Example Questions for the Benchmark

Category	Example Question	Paraphrase 1	Paraphrase 2
Ethics	"Is it ethical to use AI for hiring?"	"Should companies use AI to decide who gets hired?"	"Is there a moral issue with AI screening candidates?"
Creative	"Write a one-sentence story about a robot learning to paint."	"In one sentence, tell a story of a robot discovering painting."	"Compose a one-liner about an automaton taking up art."

Pick YOUR OWN categories, medical, legal, coding, history, psychology...

The more varied across the room, the better the discussion

Task 1: Submit your questions and paraphrases

Example Questions for the Benchmark

Same question, 4 different prompting strategies, see how outputs differ: A store sells a jacket for $120 after a 25% discount. What was the original price?

Strategy	What You Send	Example Output
Zero-shot	Just the question	"The original price was $160."
Few-shot	3 worked examples + question	"Following the pattern: $120 / 0.75 = $160."
Chain-of-Thought	Question + "Let's think step by step."	"Step 1: Sale price = 75% of original. Step 2: $120 = 0.75 × X. Step 3: X = $160."
CoT + Self-Consistency	Run CoT 5 times, majority vote	Runs: [$160, $160, $160, $150, $160] → Majority: $160

Example Questions for the Benchmark

Strategy	What You Send	Example Output
Zero-shot	Just the question	"The original price was $160."
Few-shot	3 worked examples + question	"Following the pattern: $120 / 0.75 = $160."
Chain-of-Thought	Question + "Let's think step by step."	"Step 1: Sale price = 75% of original. Step 2: $120 = 0.75 × X. Step 3: X = $160."
CoT + Self-Consistency	Run CoT 5 times, majority vote	Runs: [$160, $160, $160, $150, $160] → Majority: $160

CoT gives the reasoning path, Self-Consistency catches occasional errors.

Your task will use YOUR chosen dataset, results will vary!

Suggested Datasets for the Ablation (Pick One)

Task	Dataset	Size to Use	How to Load	Why It's Good
Math word problems	GSM8K	50–100	`load_dataset("gsm8k", "main", split="test[:50]")`	Clear answers, varying difficulty
Sentiment analysis	SST-2	50–100	`load_dataset("glue", "sst2", split="validation[:50]")`	Binary labels, easy to score
Reading comprehension	SQuAD 2.0	50	`load_dataset("squad_v2", split="validation[:50]")`	Extractive QA, good answers

Suggested Datasets for the Ablation (Pick One)

Task	Dataset	Size to Use	How to Load	Why It's Good
Commonsense reasoning	HellaSwag	50	`load_dataset("hellaswag", split="validation[:50]")`	Multiple choice, easy accuracy
Code generation	HumanEval	20	`load_dataset("openai_humaneval", split="test[:20]")`	Has test cases for verification
Trivia / factual	TriviaQA	50	`load_dataset("trivia_qa", "rc", split="validation[:50]")`	Short answers, easy exact match

Task 1: Add your dataset and prompting strategies

Step 1 Research Planning with AI

Research Planning Prompts

Project 1: I want to evaluate LLM consistency on paraphrased questions. Help me design: (1) 5 question categories, (2) how to create paraphrases, (3) what metrics to measure, (4) which models to test

Project 2: "I want to run an ablation study on prompting strategies. Help me plan: (1) what task to evaluate, (2) which prompting strategies to compare, (3) what dataset to use, (4) what metrics. Use plan mode."

Cursor + Gemini (free): Open a new file `research_plan.md`, then Cmd+K:

Claude Code (terminal): use this in a bash and use a plan mode

CLAUDE.md Research Lab Notebook

```
# My Research Project

## Research Question
[Your 1-sentence question here]

## Experiment Plan
- Models: [list]
- Dataset: [description]
- Metrics: [list]

## Decisions Made
- [Date]: Chose X because Y

## Mistakes to Avoid
- [Claude will add these as you work]
```

> "After every correction, end with 'Update your CLAUDE.md so you don't
> make that mistake again.' This is your institutional memory."

TASK 1, Plan Both Experiments (10 min)

Part A: Benchmark Plan (5 min)

1. Your 5 question categories (take inspiration from the examples slide)

2. An example question + 3 paraphrases for ONE category

3. Which 2-3 models you'll test

4. Your consistency metric definition

Everyone's plans will be different, that's expected and good.

TASK 1, Plan Both Experiments (10 min)

Part B: Ablation Plan (5 min)

1. Your chosen task + dataset (see the datasets slide)

2. Your 3-4 prompting strategies with example prompts

3. Same 2-3 models from Part A

4. Your primary metric (accuracy, F1, etc.)

Everyone's plans will be different, that's expected and good.

TASK 1, Common Mistakes(1)

Mistake	Example	Fix
All questions same difficulty	5 easy math questions like “2+3=?” — all get 100% consistency, learns nothing	Mix: “2+3” **and** “integral of e^x sin(x)” in the same category
Paraphrases are just synonym swaps	“What is 15% of 200?” → “What is fifteen percent of 200?” (trivial change)	Restructure: “If you take 15% from 200, what remains?” (different framing)
No ground truth for subjective Qs	“Is AI ethical?” — two valid but different essays score as “inconsistent”	Define per type: exact match for math, theme overlap for ethics

TASK 1, Common Mistakes(2)

Mistake	Example	Fix
Testing only 1 model	GPT-4o gets 85% consistency — is that good or bad? No way to know	Compare GPT-4o (85%) vs Claude (72%) vs Gemini (90%) → now it's interpretable
Metric chosen after seeing results	You pick "semantic similarity" because it makes your numbers look better	Pre-register: "exact match for factual, cosine sim > 0.8 for open-ended"
No answer normalization	"$160" vs "The original price was $160." flagged as mismatch (they agree)	Strip to key answer first: both become "$160", then compare

Project Scaffolding

You start coding and end up with one giant main.py that does everything, loading data, calling APIs, computing metrics, making plots. When something breaks, you can't tell where. When you want to change your model, you rewrite half the file

1. Debug faster: API call failing? You know it's in model_api.py, not buried in line 347 of main.py
2. Reuse code: Both experiments share model_api.py — write it once
3. Collaborate: Your teammate works on metrics.py while you build data_loader.py
4. Reproduce: 6 months later, a reviewer asks "how did you compute consistency?" you open metrics.py, not search through 500 lines

Student ran 200 API calls ($4), saved results to a variable, then the script crashed during plotting. All results lost. With scaffolding: results save to results/raw_responses.json after the API step

Project Scaffolding

```
"Create a Python project structure for an LLM evaluation
project that includes both a consistency benchmark and a prompting
ablation study. I need:
- data/ folder for prompts and responses
- src/ folder with modules for: data loading, model API calls,
  metrics computation, and visualization
- benchmark_main.py for the consistency benchmark
- ablation_main.py for the prompting ablation
- requirements.txt
Set it up so I can run: python benchmark_main.py --model gpt-4 --dry-run"
```

The Data Modules

Your experiment data (questions, prompts, datasets) and your experiment code (API calls, metrics) are different concerns. Mixing them creates chaos

1. Change a question → edit code, hope nothing breaks but with data modules you edit JSON, code doesn't change
2. What 50 questions did I test?→ read through Python code. With data module, you open a questions.json file
3. Rerun with different questions → rewrite the script, but with a data module you would swap the JSON file
4. Reviewer asks "show me your test data” hand them the file

The Data Modules for Benchmark dataset

```
{
    "id": "math_001",
    "category": "math",
    "original": "What is 15% of 200?",
    "paraphrases": [
        "Calculate 15 percent of 200.",
        "If you take 15% from 200, what do you
get?",  "200 times 0.15 equals what?"
    ]
```

The Data Modules for Ablation prompts + dataset:

```
STRATEGIES = {
    "zero_shot": "{question}",
    "few_shot": "Examples:\n{examples}\n\nNow answer: {question}",
    "chain_of_thought": "{question}\nLet's think step by step.",
    "cot_self_consistency": "{question}\nThink step by step. (run 5x, majority vote)"
}
```

evaluate.py

What it does: Takes your questions JSON, sends each question + all its paraphrases to an LLM, and saves the raw responses.

```
Build src/evaluate.py that:
1. Loads questions from data/questions.json
2. Runs each original + all 3 paraphrases through an LLM API
3. Saves raw responses to results/raw_responses.json
4. Supports --model, --dry-run, and --mock flags
Make it work with: python benchmark_main.py --model gpt-4o-mini --dry-run"
```

evaluate.py

What it does: Takes your questions JSON, sends each question + all its paraphrases to an LLM, and saves the raw responses.

```
Build src/evaluate.py that:
1. Loads questions from data/questions.json
2. Runs each original + all 3 paraphrases through an LLM API
3. Saves raw responses to results/raw_responses.json
4. Supports --model, --dry-run, and --mock flags
Make it work with: python benchmark_main.py --model gpt-4o-mini --dry-run"
```

evaluate.py

What it does: Takes your questions JSON, sends each question + all its paraphrases to an LLM, and saves the raw responses.

```
def evaluate_question(question: dict, model: str, mock: bool = False) ->
dict:"""
    Send original + all paraphrases to the model.

    Input:  {"id": "math_001", "original": "What is 15% of 200?",
             "paraphrases": ["Calculate 15 percent of 200.", ...]}

    Output: {"id": "math_001", "model": "gpt-4o-mini",
             "original_response": "30",
             "paraphrase_responses": ["30", "30", "The answer is 30"],
             "timestamp": "2026-03-01T10:00:00Z"}
    """
```

evaluate.py

What it does: Takes your questions JSON, sends each question + all its paraphrases to an LLM, and saves the raw responses.

```
def run_all(questions: list, model: str, mock: bool, dry_run: bool) -> list:
    """
    Loop through all questions.
    - If dry_run: just print count ("Would run 80 prompts"), don't call API
    - If mock: return fake responses like "mock_answer_for_{id}"
    - Otherwise: call the real API

    IMPORTANT: Save results to results/raw_responses.json after EACH
question (so a crash at question 15 doesn't lose questions 1-14).
    """
```

evaluate.py

Key decisions to make:

- Which API library? (`openai`, `anthropic`, `google-generativeai`)

- Temperature? (use `0` for reproducibility)

- Max tokens? (same for ALL calls — e.g., `max_tokens=500`)

- Rate limits? (add `time.sleep(1)` between calls, retry on 429)

metrics.py

What it does: Compares the original response to paraphrase responses and scores how consistent they are.

```
def exact_match(response_a: str, response_b: str) -> bool:
    """
    Are the two responses the same answer after normalization?

    Normalization: lowercase, strip whitespace, remove punctuation,
    extract the key answer (e.g., "$160" from "The answer is $160.")

    Examples:
      exact_match("The answer is $160.", "$160")  → True
      exact_match("Paris", "paris")               → True
      exact_match("It's ethical", "There are concerns") → False
    """
```

metrics.py

What it does: Compares the original response to paraphrase responses and scores how consistent they are.

```python
def semantic_similarity(response_a: str, response_b: str) -> float:
    """
    How similar are the responses in meaning? Returns 0.0 to 1.0.

    Simple approach (no ML needed):
      words_a = set(response_a.lower().split())
      words_b = set(response_b.lower().split())
      return len(words_a & words_b) / len(words_a | words_b)

    Better approach: TF-IDF cosine similarity (use sklearn)

    Examples:
      semantic_similarity("The capital is Canberra", "Canberra is the capital") → ~0.95
      semantic_similarity("Yes, AI hiring is ethical", "No, AI hiring is bad")  → ~0.4
    """
```

metrics.py

What it does: Compares the original response to paraphrase responses and scores how consistent they are.

```
def consistency_score(original_response: str,
                      paraphrase_responses: list[str]) -> dict:
    """
    Compare the original against ALL paraphrase responses.

    Returns: {
      "exact_match_rate": 0.67,      # 2 out of 3 matched exactly
      "avg_similarity": 0.85,        # average semantic similarity
      "all_responses_agree": False,   # True only if ALL match
      "num_paraphrases": 3
    }

    This is THE key metric for the benchmark.
    """
```

Task 2: Run all 3 python files for this experiment

Task 2: Run all 3 python files for this experiment

```
$ python benchmark_main.py --model [model name like gpt-4o-mini]
Loading 20 questions from data/questions.json...
Running question 1/20: math_001 — "What is 15% of 200?"
  Original response: "30"
  Paraphrase 1: "30"
  Paraphrase 2: "30"
  Paraphrase 3: "The answer is 30"
Running question 2/20: legal_001 — "Is jaywalking a felony?"
  Original response: "No, jaywalking is typically a minor
infraction..."
  Paraphrase 1: "Jaywalking is generally classified as..."
  Paraphrase 2: "No, it is not a felony..."
  Paraphrase 3: "In most jurisdictions, jaywalking is a civil..."
...
Done! 80 prompts completed. Results saved to results/raw_responses.json
Average consistency: 78.5% exact match, 0.87 semantic similarity
```

ABLATION STUDIES PROJECT

evaluate.py

```
claude "Build ablation_evaluate.py that:
1. Loads dataset from HuggingFace (or data/test_examples.json)
2. Runs each example through ALL prompting strategies
3. Saves results as (example × strategy × model) to
results/ablation_*.jsonl
4. Supports --model, --strategy, --dry-run, --mock flags
Make it work with: python ablation_main.py --model gpt-4o-mini --dry-
run"
```

evaluate.py what does it contain?

```
def format_prompt(question: str, strategy: str, examples: list = None) ->
str:"""
    Apply the strategy template to a question.

    Examples:
      format_prompt("What is 25% of 80?", "chain_of_thought")
      → "What is 25% of 80?\n\nLet's think step by step."

      format_prompt("What is 25% of 80?", "few_shot", examples=[...])
      → "Here are some examples:\n...\n\nNow solve: What is 25% of 80?"
    """
```

evaluate.py what does it contain?

```python
def extract_answer(response: str, task_type: str) -> str:
    """
    Parse the final answer from a model's response.

    WITHOUT this, "Step 1: 25% = 0.25. Step 2: 0.25 × 80 = 20. Answer: 20"
    won't match the expected answer "20".

    Strategies:
    - Look for "Answer: X" or "the answer is X" patterns
    - Take the last number in the response (for math tasks)
    - For sentiment: look for "positive" / "negative"

    Examples:
      extract_answer("Step 1: ... Step 2: ... The answer is 20.", "math") → "20"
      extract_answer("The sentiment is positive.", "sentiment")           → "positive"
    """
```

evaluate.py what does it contain?

```python
def run_all(dataset: list, strategies: list, model: str,
            mock: bool, dry_run: bool) -> list:
    """
    Nested loop: for each example × each strategy → run_single().

    50 examples × 4 strategies = 200 API calls
    (CoT-SC runs 5x internally, so ~250 extra for that strategy)

    Save incrementally to results/ablation.json after each example.
    """
```

evaluate.py

Key decisions to make:

- Same temperature for ALL strategies (`temperature=0`)

- Same `max_tokens` for ALL strategies (e.g., 500) — don't give CoT more room

- Separate few-shot examples from test data (data contamination!)

- Log exact model ID and timestamp in every resul

ablation_metrics.py

```python
def accuracy(results: list[dict]) -> float:
    """
    What fraction of answers were correct?

    Example: 39 out of 50 correct → 0.78 (78%)
    """

def per_strategy_accuracy(all_results: list[dict]) -> dict:
    """
    Group results by strategy, compute accuracy for each.

    Returns: {
      "zero_shot":             {"accuracy": 0.62, "correct": 31, "total": 50},
      "few_shot":              {"accuracy": 0.72, "correct": 36, "total": 50},
      "chain_of_thought":      {"accuracy": 0.78, "correct": 39, "total": 50},
      "cot_self_consistency":  {"accuracy": 0.82, "correct": 41, "total": 50}
    }

    This is THE key output for the ablation study.
    """
```

ablation_metrics.py

```python
def statistical_comparison(scores_a: list[bool],
                           scores_b: list[bool]) -> dict:
    """
    Are two strategies significantly different?

    Use McNemar's test (or paired t-test on per-example 0/1 scores).

    Returns: {
      "strategy_a_accuracy": 0.72,
      "strategy_b_accuracy": 0.78,
      "difference": 0.06,
      "p_value": 0.034,
      "significant": True    # p < 0.05
    }

    If p < 0.05 → "The difference is statistically significant."
    If p >= 0.05 → "We cannot conclude the strategies differ."
    """
```

Quick reference table

File	Input	Output	Key Function
`data/questions.json`	—	20 questions with paraphrases	Your experiment data
`src/evaluate.py`	`questions.json`	`results/raw_responses.json`	Send questions to LLM APIs
`src/metrics.py`	`raw_responses.json`	Consistency scores	Compare original vs paraphrases
`src/prompts.py`	—	Strategy templates	Define zero-shot, few-shot, CoT, CoT + SC
`src/ablation_evaluate.py`	Dataset + prompts	`results/ablation.json`	Run all strategies on all examples
`src/ablation_metrics.py`	`ablation.json`	Accuracy per strategy	Score and compare strategies

Task 2: Run all 3 python files for this experiment

```
$ python ablation_main.py --model gpt-4o-mini --strategy all
Loading 50 examples from GSM8K...
Running example 1/50 with zero_shot: "Natalia sold clips..."
  Response: "72"  Expected: "72"  ✓
Running example 1/50 with few_shot: "Natalia sold clips..."
  Response: "72"  Expected: "72"  ✓
Running example 1/50 with chain_of_thought: "Natalia sold
clips..."
  Response: "Step 1: 48/2=24... Answer: 72"  Extracted: "72"  ✓
...
Done! 200 prompts completed. Results saved to
results/ablation.json
Strategy Results:
  zero_shot:            62% (31/50)
  few_shot:             72% (36/50)
  chain_of_thought:     78% (39/50)
  cot_self_consistency: 82% (41/50)
```